

Day 1

thinking in algorithms

Algorithm to define addition of two number

start

int a, b

int a = 10;

int b = 20;

int c = a + b;

end

complexity

Name

$O(1)$

constant

$O(\log n)$

logarithmic

$O(n)$

linear

$O(n \log n)$

linearithmic

$O(n^2)$

Quadratic

$O(2^n)$

Exponential

$O(n!)$

Factorial

Day 1

thinking in algorithms
 == == ==

Algorithm to define addition of two number

start

int a, b

int a = 10;

int b = 20;

int c = a + b;

end

complexity

Name

$O(1)$

constant

$O(\log n)$

logarithmic

$O(n)$

linear

$O(n \log n)$

linearithmic

$O(n^2)$

Quadratic

$O(2^n)$

Exponential

$O(n!)$

Factorial

Day 1

thinking in algorithms

Algorithm to define addition of two number

start

int a, b

int a = 10;

int b = 20;

int c = a + b;

end

complexity

Name

 $O(1)$

constant

 $O(\log n)$

logarithmic

 $O(n)$

linear

 $O(n \log n)$

linearithmic

 $O(n^2)$

Quadratic

 $O(2^n)$

Exponential

 $O(n!)$

Factorial

single $\rightarrow O(n)$

nested $\rightarrow O(n^2)$

triangle $\rightarrow O(n^2)$

halving $\rightarrow O(\log n)$

$n \log n \rightarrow O(n \log n)$

sequential $\rightarrow O(n)$

sqr loop $\rightarrow O(n)$

sqr loop $\rightarrow O(\sqrt{n})$

constant $\rightarrow O(1)$

Day 2 - ADS using Java

27/9/26

In Java we can't get exact address
bcz java does not support the pointers

array marks = 1000 1004 1008 1012 1016 1020
marks = 12 45 7 33 96 18

0 1 2 3 4 5

address of marks [3] = $1000 + 3 \times 4 = 1012$

one multiplication and one addition
whatever size of array $O(1)$

array index $\rightarrow O(1)$

```
static void insertAt (int[] a, int size,
                     int index, int value) {
```

```
    for (int i = size; i > index; i--) {
```

```
        a[i] = a[i-1];
```

```
    }
    a[index] = value;
```

```
}
```

⇒ insertAt (a, 4 elements, insert 15 at index 1);

a = [10, 20, 30, 40, 0]

5 position → because we need one empty space for new element

We want value 15 at index 1

Answer should be

[10, 15, 20, 30, 40]

```
for (int i = size; i > index; i--) {
```

size = 4, index = 1

i = 4, 4 > 1 → Yes, 4 → 3 → 2, 0

When i = 1, 1 > 1, No

So i goes 4, 3, 2

first iteration

$$a[i] = a[i-1]$$

$i=4$

$$a[4] = a[3]$$

index :- 0 1 2 3 4

10 20 30 40 0

copy 40

$$a[4] = a[3]$$

10 20 30 40 40

second :- $i=3$

==

$$a[3] = a[2]$$

30 $\rightarrow$ index 3

10 20 30 30 40

ind :- $i=2$

$$a[2] = a[1]$$

10 20 20 30 40

now $a[index] = value$

$$a[1] = 15$$

0, 15, 20, 30, 40 ✓ done

ay. copy of() $\rightarrow$ for create big array and copy the old elements

void grow() → make array

bigger

M T W T F S S	
Page No.:	YOUVA
Date:	

code

dynamic array

```
public void add(T item) {  
    if (size == data.length) grow();  
    data[size++] = item;  
}
```

y

```
public void grow() {  
    int oldcap = data.length;  
    int newcap = Math.max(oldcap + (oldcap  
        > 1 ? 1 : 0), oldcap  
        + 1);  
    data = Arrays.copyOf(data, newcap);  
}
```

y

data = {10, 20, 30, 40}

size = 4

index = 1

data[1] = data[2]

10, 20, 30, 40 → 10 30 30 40

0 1 2 3

100p ②

1/2

100p ② 1/2

data[2] = data[3]

[10, 30, 40, 40]

size--;

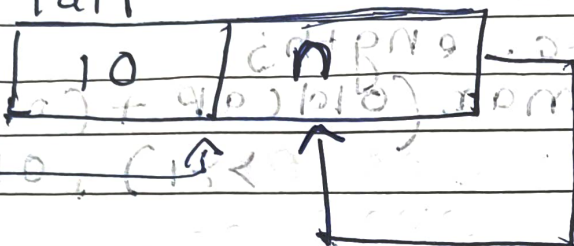
10, 30, 40

if (size == 0) return 0;

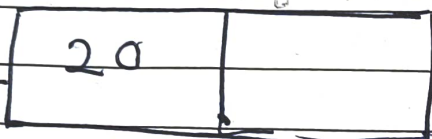
return data[size-1];

Circular link list

Tail



n->next = 0



[0, 0, 0, 0] = 0

size = 0

i = 0

[0] = 0

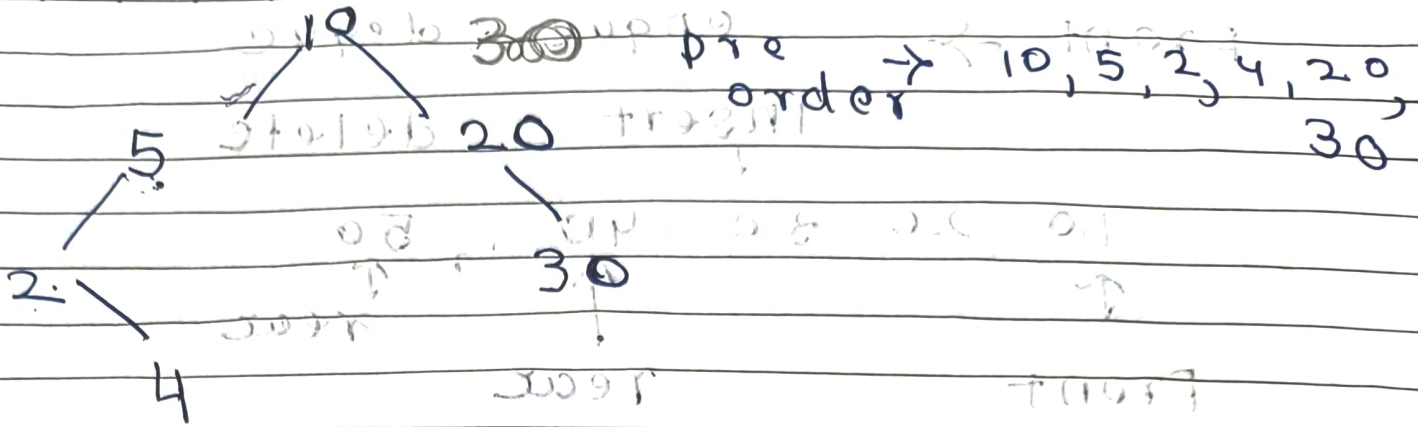
0 0 0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0

Tree

10, 5, 2, 4, 20, 30

M T W T F S S						
Page No.:				YOUVA		
Date:						



Inorder (left, root, right)

5, 2, 4, 10, 20, 30

postorder → 4, 2, 5, 30, 20, 10

preorder → 10, 5, 2, 4, 20, 30

preorder (root, left, right)

Array $\rightarrow$ O(1)

queue

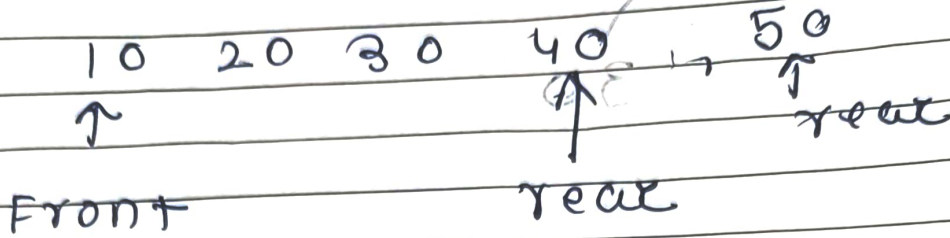
rear $\rightarrow$

Front $\rightarrow$

enqueue

dequeue

insert $\rightarrow$ delete



enqueue $\rightarrow$ add at rear

Dequeue $\rightarrow$ remove from Front

enqueue $\rightarrow$ rear

dequeue $\rightarrow$ Front

peek $\rightarrow$ front

5 10 15

10 20 30

Front rear

5 10 15

20

Front

30

20

20

20

20

10

10

rear formula
 $rear = (front + size - 1) \% capacity$

capacity;

next free position $\rightarrow (rear + 1) \% capacity$

$$(5 + 5 - 1) \% 10 = 9$$

9, 1, 8

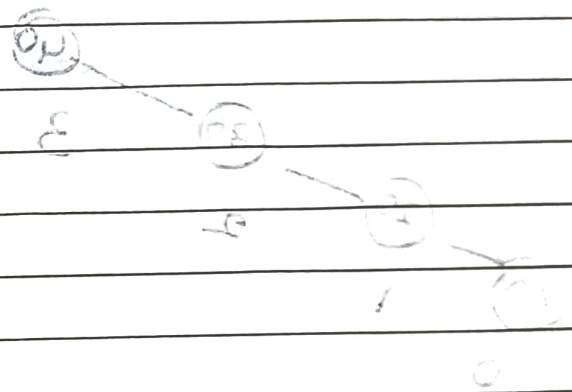
8

1

$$(4 + 1) \% 5 = 0$$

$$(4 + 3 - 1) \% 5 = 6$$

Circular queue



0
 1
 2
 3
 4
 5
 6
 7
 8
 9

front + size - 1
 modulo capacity

10
 20
 30
 40
 50
 60
 70
 80
 90
 100

Day 6
==

AVL tree

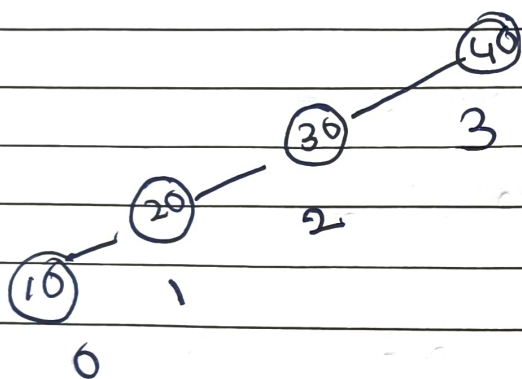
AVL tree is self balancing binary search tree

BST rule $\rightarrow$ left child is smaller than the parent right child is greater

Balance rule $\rightarrow$ height diff between left and right subtree of every node cannot be more than 1

Balance Factor = $H(\text{Left}) - H(\text{Right})$

-1, 0, +1



Four type of AVL rotation when node become unbalanced

LL

RR

LR

RL

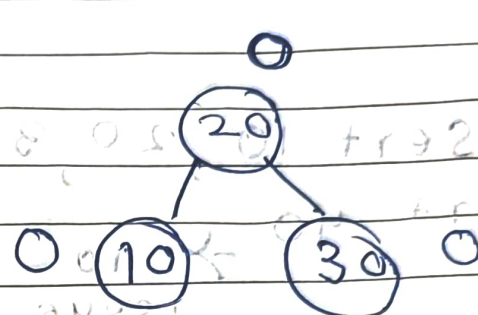
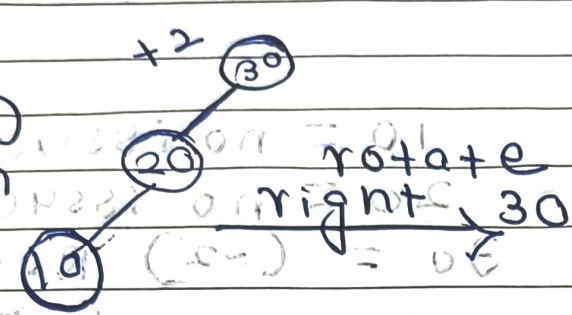
LL Rotation (Left Rotation)

RR Rotation (Right)

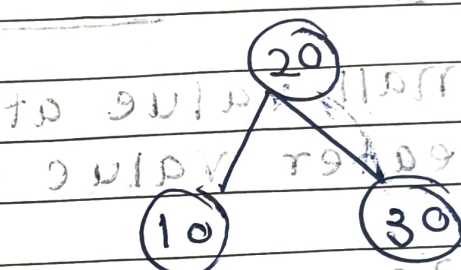
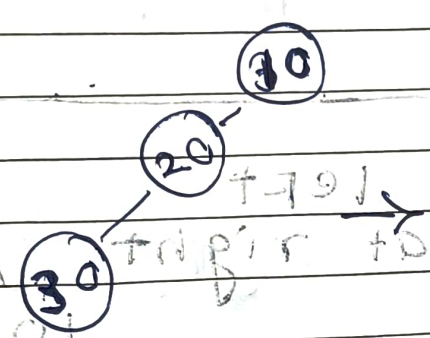
LR more than 1 rotation 1st left and then right

RL more than one rotation 1st right and then left

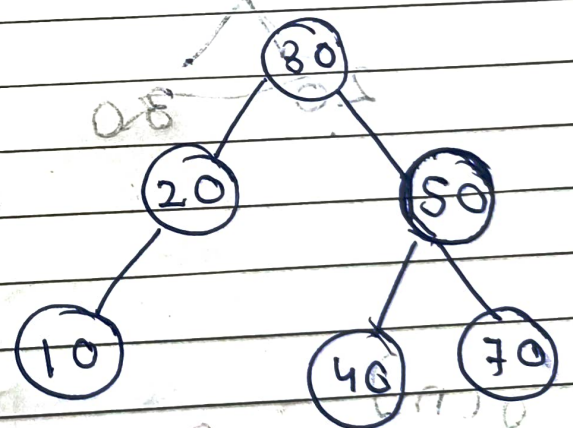
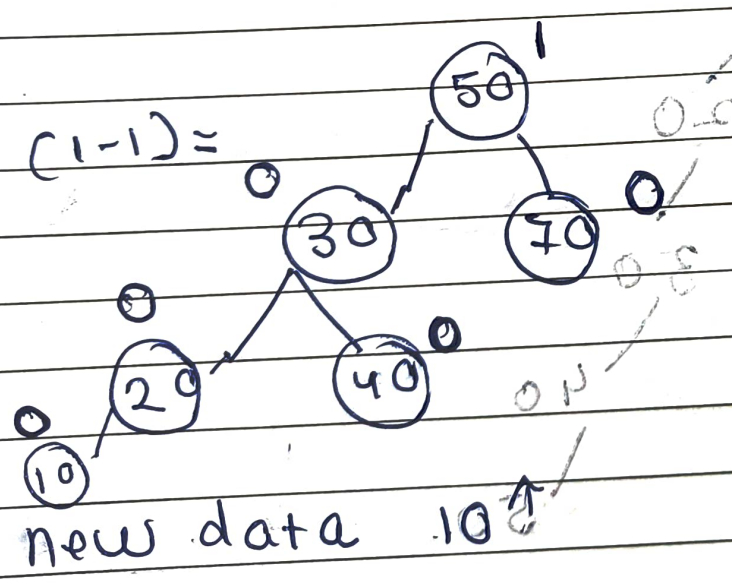
LL
(Right) rotation



RR
(Left rotation)



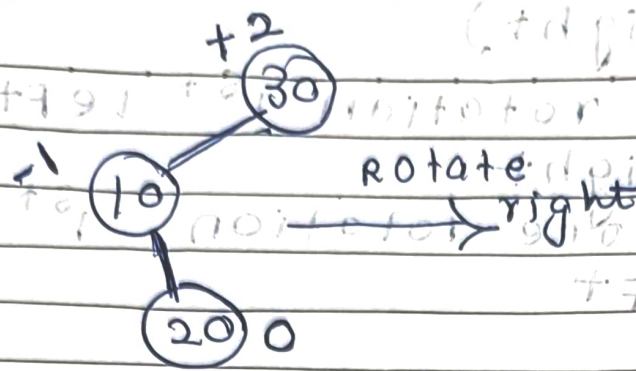
Insert 50, 30, 70, 20, 40



LR

insert 30 10 20

Page No.:	YOUVA
Date:	



Q

insert 10, 20, 30

insert 40

no issue

10 = no issue

20 = no issue

30 = (-2) Rotation

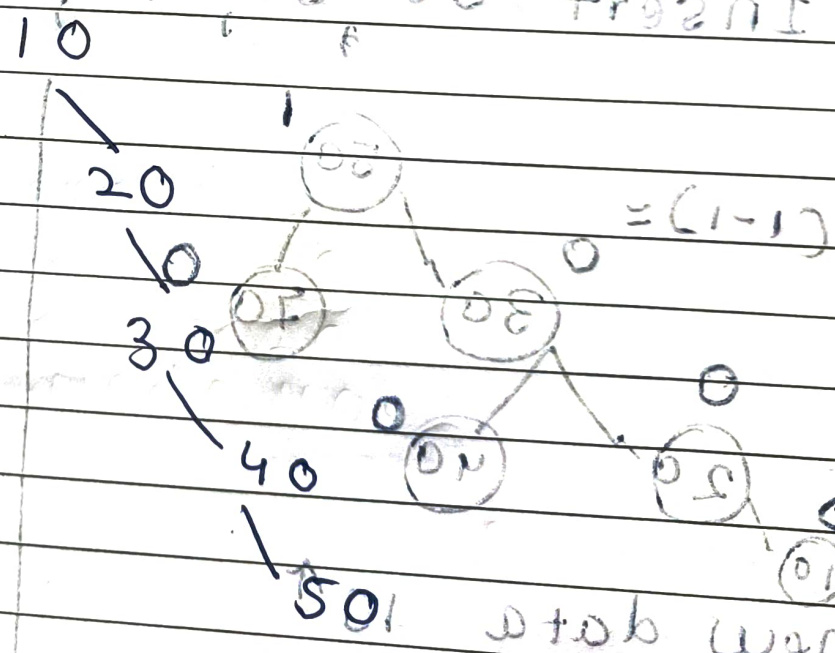
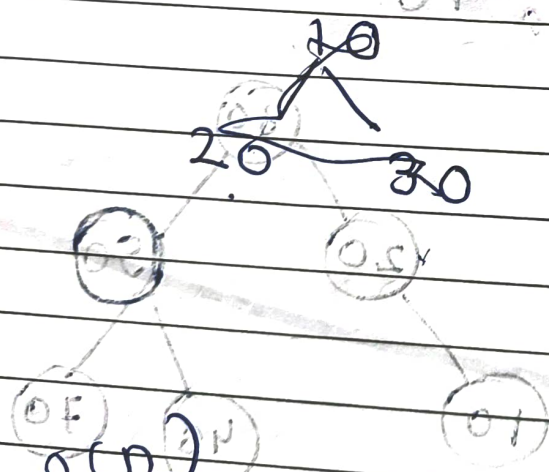
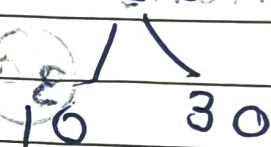
left RR case

so .

BST

small value at left
Greater value at right

10, 20, 30, 40, 50



$O(n)$
 $O(\log n)$

$+ \rightarrow$ left heavy
 $- \rightarrow$ right heavy

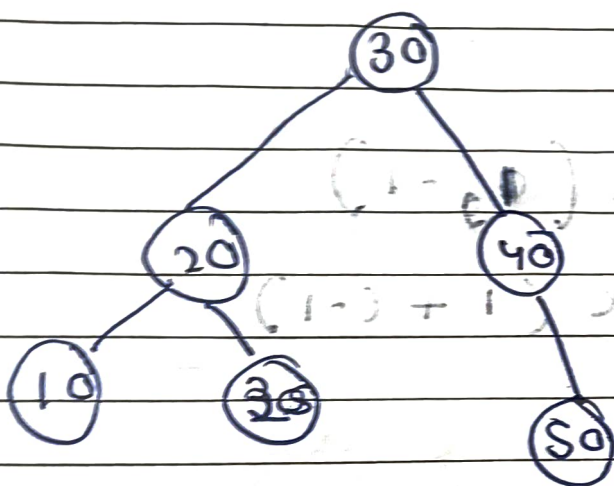
$+2 \rightarrow$ too much left
 $-2 \rightarrow$ too much right

AVL $\rightarrow$ Adelson velsky and landis

Balance Factor = height(left) - height(right)

Empty subtree has height -1

leaf has height 0



10 has no children

left = empty

right = empty = -1

$height = 1 + \max(\text{left height}, \text{right height})$

$1 + \max(-1, -1)$

$1 + (-1)$

② 20 $\rightarrow$ 10 height = 0

10 left = 0 right = -1

height(20)

$1 + \max(0, 1)$

$1 + 0$

$= 1$

height ①

height 0

30

20

$\Rightarrow$

10 $\rightarrow$ height 0

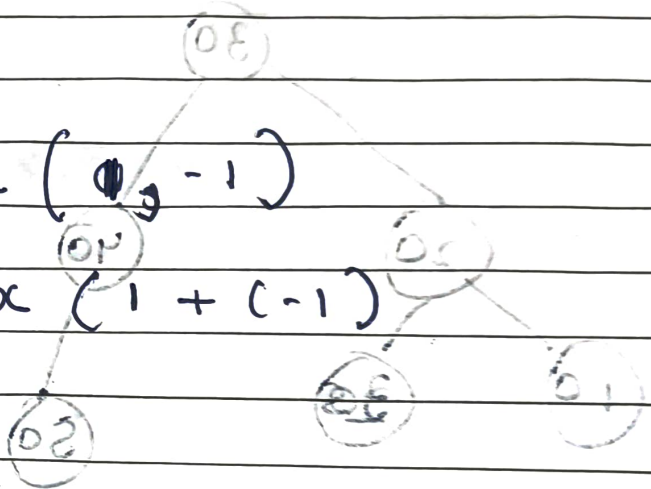
20 $\rightarrow$ height 1

30 $\rightarrow$

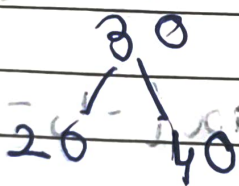
height(30)

$= 1 + \max(0, 1)$

$1 + \max(1, 1)$



suppose



There are 3 node

Node 30

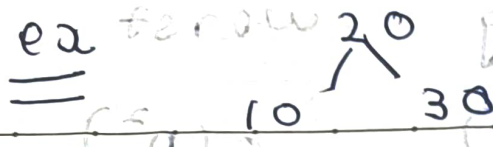
Node 20

Node 40

class Node {

int key, height;

Node left, right;



① $h(10) = 0$ 10 is a leaf

② $h(30) = 0$ 30 is a leaf

③ $balance(20) = \text{left height} - \text{right height}$

$$= 0 - 0 = 0$$

④ $height(20) = 1 + \max(h(n.\text{left}), h(n.\text{right}))$

$$= 1 + \max(h(10), h(30))$$

$$= 1 + \max(0, 0)$$

$$= 1 + 0$$

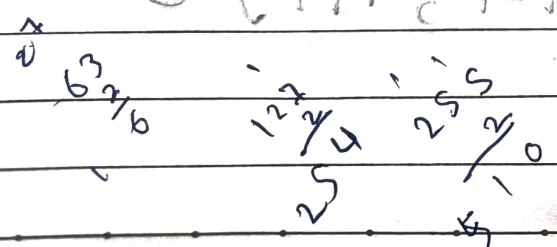
$$= 1$$

max node = 2^h

max node in a complete binary tree of height 5 (edge)

$$\Rightarrow 2^{h+1} \text{ pairs} \Rightarrow 2 = 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 63$$

$$2^0, 2^1, 2^2, 2^3, 2^4, 2^5 = 1, 2, 4, 8, 16, 32$$



Algorithm

Best

Avg

Worst

Page No.:

YOUVA

Date:

Selection

$O(n^2)$

$O(n^2)$

$O(n^2)$

Insertion

$O(n^2)$

$O(n^2)$

$O(n^2)$

Bubble

$O(n^2)$

$O(n^2)$

$O(n^2)$

Merge

$O(n \log n)$

$O(n \log n)$

$O(n \log n)$

Quick

$O(n \log n)$

$O(n \log n)$

$O(n^2)$

Heap

$O(n \log n)$

$O(n \log n)$

$O(n \log n)$

Day 7

Sorting

*

Selection Sort

Bubble

{4, 5, 2, 3}

compare adjacent element

code

class Sorting {

public static void main (String[] args)

int arr[] = {7, 8, 3, 1, 2};

for (int i=0; i < arr.length-1; i++) {

For (int j=0; j < arr.length-1; j++)
 if (arr[j] > arr[j+1]) {

// swap

int temp = arr[j];
 arr[j] = arr[j+1];
 arr[j+1] = temp;

★ selection sort ★

[4, 8, 3, 1, 2]

for (int i=0; i < arr.length; i++) {
 int smallest = i;

for (int j=i+1; j < arr.length; j++) {

if (arr[smallest] > arr[j]) {

arr[smallest] = arr[j];

smallest = j;

int temp = arr[smallest];
 arr[smallest] = arr[i];
 arr[i] = temp;

① 400

② 110

③ 100

④ 010

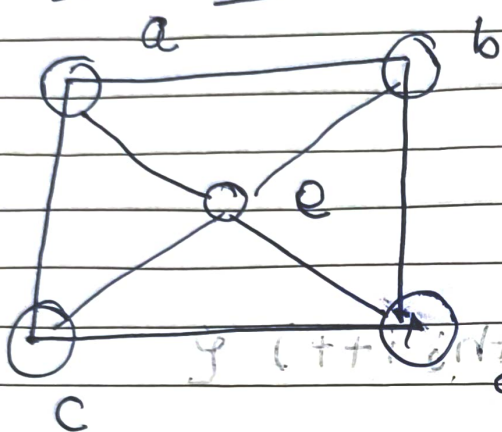
⑤ 110

Insertion sort

Graph

A graph is (V, E) is set of vertices and edges where we write $|V| = V$ and $|E| = E$

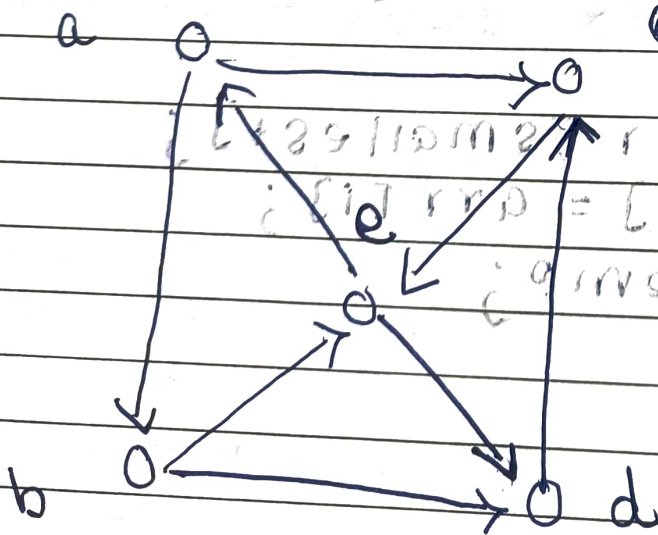
undirected (edge work both way)



	a	b	c	d	e
a	0	1	1	0	1
b	1	0	0	1	1
c	1	0	0	1	1
d	0	1	1	0	1
e	1	1	1	1	0

degree of a, b, c, d = 3 = 6 (1+1+1) 107

directed graph



a → in ①

out ②

c → in ②

out ①

d → in ②

out ①

e → in ①

out ②

b → in ①

	a	b	c	d	e
a	0	1	1	0	0
b	0	0	0	1	1
c	0	0	0	0	1
d	0	0	1	0	0
e	0	0	1	0	0

★ Selection Sort

```

for (int i=0; i < a.length-1; i++) {

```

```

    int min = i;

```

```

    for (int j=i+1; j < a.length; j++) {

```

```

        if (a[j] < a[min]) {

```

```

            min = j;

```

```

        int temp = a[i];

```

```

        a[i] = a[min];

```

```

        a[min] = temp;

```

int a[] = { 6, 3, 8, 1, 5 }

min = i = 0 = 6

3 < 6
 yes
 min = 3
 8 < 3
 no
 1 < 3
 yes
 min = 1

Insertion sort

5, 3, 4

```
import java.util.*;

public class main {

    static void insertionSort (int[] a) {
        for (int i=1; i < a.length; i++) {
            int key = a[i];
            int j = i-1;
            while (j >= 0 && a[j] > key) {
                a[j+1] = a[j];
                j = j-1;
            }
            a[j+1] = key;
        }
    }
}
```

element 5 is already sorted

key = a[i] = a[1] = 3

[5] [3] [4]

↑
key = a[i] = 3
j = i-1 = 0-1 = -1

key = 3

5 3 4
5 3 4 = 5 3 4
a = 0 = 1 = min